5.1 Array Methods

---

1) How does `map()` differ from `forEach()`?

- `map` returns a **new array** of the same length (transformation).
- `forEach` iterates for side effects and returns `undefined`.

```js
const arr = [1,2,3];
const doubled = arr.map(x => x * 2);   // [2,4,6]
const out = arr.forEach(x => x * 2);   // undefined
```

Tip: Prefer `map` when you need a result; `forEach` for logging, DOM ops, etc.

---

2) Can `map()` be chained with `filter()` / `reduce()`? Perf implications?

Yes. Order matters: put `filter` → `map` (avoid transforming items you'll drop).

```js
const emails = users
  .filter(u => u.active)
  .map(u => u.email.toLowerCase());
```

Perf: Each pass creates an intermediate array. For huge data, consider a **single** `reduce` or a transducer-style pipeline.

---

3) `reduce()` vs `reduceRight()`

Both aggregate; `reduceRight` processes from **right to left**.

```js
['a','b','c'].reduce((a,b)=>a+b);      // "abc"
['a','b','c'].reduceRight((a,b)=>a+b); // "cba"
```

Useful for right-associative ops (e.g., function composition).

4) `map()` on sparse arrays (holes)?

Holes are **preserved**; callback **is not called** for holes.

```js
const a = [1, , 3];
a.map(x => x*2); // [2, , 6]
```

Prefer: normalize first (`Array.from(a)` or `a.flat()`).

---

5) Flatten nested arrays with `reduce()`

```js
const flat = (arr) =>
  arr.reduce((acc, v) => acc.concat(Array.isArray(v) ? flat(v) : v), []);
flat([1,[2,[3]]]); // [1,2,3]
```

(For one level: `arr.flat(1)`; arbitrary: `arr.flat(Infinity)`.)

---

6) Polyfill `map()`

```js
Array.prototype.myMap = function(cb, thisArg){
  const out = new Array(this.length);
  for (let i=0; i<this.length; i++){
    if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  }
  return out;
};
```

---

7) Group objects by a property (with `reduce`)

```js
```

```
const byDept = employees.reduce((acc, e) => {
  (acc[e.dept] ||= []).push(e);
  return acc;
}, {});
```

---

8) `some()` vs `every()`

- `some` → at least one element satisfies predicate.
- `every` → all elements satisfy predicate.

  Both **short-circuit**.

```
[1,3,5].some(x=>x%2===0);  // false
[2,4,6].every(x=>x%2===0); // true
```

---

9) Remove duplicates

```
const unique1 = [...new Set(arr)];
const unique2 = arr.filter((v,i) => arr.indexOf(v) === i);
```

Best: `Set` for primitives. For objects, dedupe by key (`Map` / `Set` on `id`).

---

10) `find()` vs `findIndex()` + complexity

- `find` returns the **first matching value**.
- `findIndex` returns the **index** (or `-1`).

  Both **O(n)** worst-case.

```
users.find(u => u.id===42);
users.findIndex(u => u.id===42);
```

11) `flat()` vs `reduce` for flattening

- `flat(depth)` is optimized C++ path in engines; concise.
- `reduce` is flexible for *custom* flatten rules (filtering, mapping while flattening).

---

12) Returning `undefined` in `map()` callback

It **keeps the slot** with `undefined`.

```js
[1,2,3].map(x => (x>1? x: undefined)); // [undefined,2,3]
```

If you want to remove, use `filter` first or a single `reduce`.

---

13) Use `reduce()` to implement `map()`

```js
const mapped = arr.reduce((acc, v, i, a) => {
  acc.push(fn(v, i, a));
  return acc;
}, []);
```

Useful to avoid extra passes when combining logic.

---

14) `Array.of()` vs `Array.from()` vs `new Array()`

- `Array.of(3)` → `[3]` (single element).
- `new Array(3)` → empty array with **length 3 (holes)**.
- `Array.from(iterable, mapFn?)` → from iterable/array-like (optionally mapping).

```js
Array.from({length:3}, (_,i)=>i); // [0,1,2]
```

---

15) Why `forEach()` not chainable?

It returns `undefined` by spec. To "chain", just use `map` / `filter` or write:

```js
Array.prototype.tap = function(fn){ this.forEach(fn); return this; };
```

---

16) Frequency count with `reduce()`

```js
const freq = arr.reduce((m, v) => (m[v] = (m[v]||0)+1, m), {});
```

---

17) `splice()` vs `slice()`

- `splice(start, deleteCount, ...items)`: **mutates** array, can insert/delete.
- `slice(start, end)`: **non-mutating** copy.

```js
const a = [1,2,3]; a.slice(0,2); // [1,2], a stays
a.splice(1,1); // removes 2, a becomes [1,3]
```

---

18) `filter()` to implement `compact()` (remove falsy)

```js
const compact = arr => arr.filter(Boolean);
compact([0,1,false,2,"",3]); // [1,2,3]
```

---

19) `reduce` for sum vs `for` loop (perf)

- `for` is typically **fastest** (no callback allocations).
- `reduce` is expressive; overhead may matter on huge arrays/hot paths.

```js
let s=0; for (let i=0;i<arr.length;i++) s+=arr[i];
```

Guideline: use `reduce` for readability unless profiling shows a hotspot.

---

20) Optimize heavy chains (`map` → `filter` → `reduce`)

- Reorder: **filter early**, then map.
- Fuse into a single `reduce` to avoid intermediate arrays.
- Consider **lazy iteration** (generators) or libraries (transducers) for massive data.

```js
const result = arr.reduce((acc, x) => {
  if (x.active) acc.push(x.value * 2);
  return acc;
}, []);
```

Quick Best-Practices Recap

- Prefer **immutable** methods (`map`, `filter`, `slice`, `concat`) in UI state.
- **Short-circuit** where possible (`some`, `every`, `find`).
- For **huge arrays**, avoid multiple passes; **combine** logic or profile.
- Use `Set` / `Map` for dedupe/grouping by keys.
- Understand **sparse arrays** and avoid holes for performance.

2️⃣ Iterators & Generators — Answers + Examples

1) What is an iterator vs an iterable?

- **Iterable**: an object that implements `Symbol.iterator` returning an **iterator**.
- **Iterator**: an object with `.next()` returning `{ value, done }`.

```js
const iterable = [1,2,3];                  // built-in iterable
const iter = iterable[Symbol.iterator](); // iterator
iter.next(); // { value: 1, done: false }
```

---

2) How does `for...of` work internally?

It calls `obj[Symbol.iterator]()` to get an iterator and repeatedly calls `.next()` until `{done:true}`.

```js
for (const x of [10,20]) {/* uses iterator under the hood */}
```

---

3) What does `Symbol.iterator` do?

Defines how an object is iterated.

```js
const range = {
  from: 1, to: 3,
  [Symbol.iterator]() {
    let cur = this.from;
    return {
      next: () => ({ value: cur, done: cur++ > this.to })
    };
  }
};
[...range]; // [1,2,3]
```

---

4) Implement a custom iterator for an object

```js
const obj = { a: 10, b: 20 };
obj[Symbol.iterator] = function*(){  // generator (iterator + iterable)
  for (const k of Object.keys(this)) yield [k, this[k]];
};
for (const [k,v] of obj) {/* ... */}
```

---

5) What is `.next()`?

Advances the iterator; returns `{ value, done }`.

```js
const it = "hi"[Symbol.iterator]();
it.next(); // { value: 'h', done: false }
it.next(); // { value: 'i', done: false }
it.next(); // { value: undefined, done: true }
```

---

6) Iterator vs Generator?

- **Generator** (`function*`): produces an **iterator** automatically; supports `yield`, pausing, `return`, `throw`, and bidirectional messaging.
- **Iterator**: any object with `.next()`.

---

7) How do generators maintain state?

They suspend at each `yield`, preserving the stack frame (locals, try/catch state) until resumed.

```js
function* counter(){ let i=0; while(true) yield ++i; }
const c = counter();
c.next().value; // 1, then 2, ...
```

8) What does `yield` do?

Pauses the generator and outputs a value; next `.next()` resumes **after** the `yield`.

```js
function* g(){ const x = yield 1; return x*2; }
const it = g();
it.next();       // { value: 1, done: false }
it.next(7);      // pass 7 back into `x` -> { value: 14, done: true }
```

9) Infinite sequence with a generator

```js
function* naturals(){ let n=0; while(true) yield n++; }
const it = naturals();
it.next().value; // 0,1,2, ...
```

Use carefully; always break or limit consumption.

10) Pause/resume async workflows with generators (co-style)

```js
function run(genFn){
  const it = genFn();
  const step = (last) => {
    const { value, done } = it.next(last);
    if (done) return Promise.resolve(value);
    return Promise.resolve(value).then(step); // chain Promises
  };
  return step();
}

run(function*(){
  const user = yield fetch('/api/user').then(r=>r.json());
```

```
  const posts = yield fetch(`/api/u/${user.id}/posts`).then(r=>r.json());
  return { user, posts };
});
```

(Conceptually how `async/await` was polyfilled before ES2017.)

---

11) What happens calling `.next()` after completion?

Always returns `{ value: undefined, done: true }` (idempotent).

---

12) Delegation with `yield*`

```js
function* letters(){ yield* ['a','b']; yield* 'cd'; }
[...letters()]; // ['a','b','c','d']
```

`yield*` delegates iteration to another iterable/generator.

---

13) Generators and lazy evaluation

Values are produced **on demand**, reducing memory.

```js
function* map(iter, fn){ for (const x of iter) yield fn(x); }
const doubled = map(naturals(), n=>n*2);
doubled.next(); // compute only when asked
```

---

14) Generator vs async/await under the hood

- Generators yield **values/promises**; require a **runner** to await.
- `async/await` is built-in syntax sugar that returns **Promises** and handles scheduling automatically.

---

15) Returning a final value from a generator

Use `return` (not yielded to `for...of`, but accessible via `.next()` result).

```js
function* g(){ yield 1; return 42; }
const it = g();
it.next();      // { value:1, done:false }
it.next();      // { value:42, done:true }
```

---

16) Iterating trees (DFS) with a generator

```js
function* dfs(node){
  if (!node) return;
  yield node.value;
  yield* dfs(node.left);
  yield* dfs(node.right);
}
```

---

17) `for...of` with generators

`for...of` consumes `yield`ed values until `done:true`.

```js
function* g(){ yield 1; yield 2; }
for (const v of g()) console.log(v); // 1, 2
```

---

18) Difference among `return`, `yield`, `throw` in generators

- `yield v` → output value, pause.
- `return v` → finish with result `v`.
- `it.throw(err)` → inject error at suspension point (caught inside generator).

```js
function* g(){
  try { yield 1; }
  catch(e){ /* handle */ }
}
const it = g();
it.next();
it.throw(new Error('boom'));
```

---

19) Infinite Fibonacci generator

```js
function* fib(){
  let a=0, b=1;
  while(true){ yield a; [a,b] = [b, a+b]; }
}
[...Array(6)].map(()=>fib().next().value); // don't do this—creates new iters!
```

Correct consumption:

```js
const f = fib();
f.next().value; // 0,1,1,2,3,5,...
```

---

20) Generators for async pipelines / data streams

Use **async generators** for streaming I/O.

```js
async function* streamLines(reader){
  for await (const chunk of reader) {
    yield* chunk.toString().split('\n');
  }
}
```

```
for await (const line of streamLines(someAsyncIterable)) {
  // process line
}
```

Bonus: Async Iterators & `Symbol.asyncIterator`

Async iterable implements `Symbol.asyncIterator` returning an **async iterator** (whose `.next()` returns a **Promise**).

```js
const asyncRange = {
  from: 1, to: 3,
  [Symbol.asyncIterator]() {
    let cur = this.from;
    return {
      next: () => new Promise(res =>
        setTimeout(() => res({ value: cur, done: cur++ > this.to }), 10)
      )
    };
  }
};

for await (const n of asyncRange) console.log(n); // 1 2 3 (with delays)
```

Best-practice recap

- Prefer **generators** for **lazy, pull-based** pipelines and complex traversals.
- Use **async generators** for streaming network/file data.
- Be careful with **infinite generators**: always cap or break.
- For heavy data transforms, generators help **avoid intermediate arrays** and reduce memory.

🧩 Section 5.3 — Spread, Rest, and Destructuring (20 Q&A)

---

1. How does the spread operator differ from `Object.assign()` for shallow copying?

Answer:

Both perform shallow copies — but `Object.assign()` mutates the target, while spread creates a new one.

```js
const obj = { a: 1, b: { c: 2 } };
const copy1 = { ...obj }; // new object
const copy2 = Object.assign({}, obj);
copy1.b.c = 99;
console.log(obj.b.c); // 99 (shared ref!)
```

Best practice:

For nested data, use `structuredClone()` or deep-copy logic.

---

2. What happens when you spread an object with nested structures?

Answer:

Only the **top-level** is copied — nested references are shared (shallow copy).

```js
const user = { name: "Ava", meta: { age: 30 } };
const clone = { ...user };
clone.meta.age = 40;
console.log(user.meta.age); // 40
```

---

3. How can you use the rest parameter to handle variable-length arguments?

Answer:

Rest collects *remaining arguments* into an array.

```js
```

```
function sum(...nums) {
  return nums.reduce((a,b)=>a+b,0);
}
sum(1,2,3,4); // 10
```

💡 `arguments` is array-like; rest is a **true array**.

---

4. How can you destructure nested objects safely (with defaults)?

Answer:

You can assign defaults for missing values.

```js
const user = { name: "Tom", address: { city: "NY" } };
const { address: { city, zip = 10001 } } = user;
console.log(city, zip); // "NY", 10001
```

---

5. Difference between destructuring arrays vs objects?

Answer:

- Arrays: position-based
- Objects: key-based

```js
const [a,b] = [1,2];          // index order
const {x,y} = {y:1,x:2};      // key name
```

---

6. How can you swap two variables using array destructuring?

```js
let a = 1, b = 2;
[a,b] = [b,a];
```

```
console.log(a,b); // 2,1
```

Clean, concise, avoids a temp variable.

---

7. How does destructuring work with function parameters?

Answer:

You can destructure directly in parameter definitions.

```js
function greet({ name, age }) {
  console.log(`Hi ${name}, age ${age}`);
}
greet({ name: "Alice", age: 25 });
```

---

8. What happens if you destructure a property that doesn't exist?

Answer:

It's simply `undefined`.

```js
const { z } = { x: 1 };
console.log(z); // undefined
```

---

9. How can you merge two arrays immutably using spread?

```js
const a = [1,2];
const b = [3,4];
const merged = [...a, ...b]; // [1,2,3,4]
```

React tip: use this pattern for merging props or lists immutably.

---

10. Difference between shallow and deep destructuring?

Answer:

Shallow destructuring extracts one level. Deep requires nested pattern.

```js
const user = { info: { name: "Tom" } };
const { info: { name } } = user; // deep destructure
```

---

11. How do rest (`...`) and spread (`...`) differ?

Operator	Purpose	Context
Spread	Expands	Arrays, objects, function calls
Rest	Collects	Function params, destructuring

```js
function f(a, ...rest) { console.log(rest); }
f(1,2,3); // [2,3]

const a = [1,2]; const b = [...a,3]; // spread
```

---

12. How can you destructure an object returned by a function?

```js
function getUser() {
  return { id: 1, name: "Ava", age: 30 };
}
const { name, age } = getUser();
```

---

13. What's the output of destructuring `null` or `undefined`?

Answer:

Throws `TypeError` (cannot destructure nullish values).

```js
const { x } = null; // ❌ TypeError
```

💡 Safe destructuring:

```js
const { x } = obj || {};
```

---

14. Can you use computed property names in object destructuring?

Yes, but indirectly via aliasing:

```js
const key = "age";
const user = { age: 20 };
const { [key]: val } = user;
console.log(val); // 20
```

---

15. Clone arrays using spread vs `slice()`

```js
const a = [1,2,3];
const clone1 = [...a];
const clone2 = a.slice();
```

Both are shallow; spread is more modern and expressive.

---

16. Spread in function calls with multiple arguments

```js
```

```
function add(a,b,c){return a+b+c;}
const nums = [1,2,3];
add(...nums); // 6
```

Equivalent to `add(nums[0], nums[1], nums[2])`.

---

17. Difference between rest parameters and `arguments` object?

Feature	Rest	`arguments`
Type	Array	Array-like
Works with arrow funcs	✅ Yes	❌ No
Supports spread methods	✅ Yes	❌ No

```js
function f(...args){ console.log(args); } // true array
```

---

18. Use destructuring for function return tuples

```js
function useValue(){
  return [10, () => console.log("updated")];
}
const [val, setVal] = useValue();
```

✅ React's `useState` uses this pattern.

---

19. How does destructuring help in React hooks?

Answer:

Hooks return tuples or objects; destructuring improves readability.

jsx

Copy code

```jsx
const [count, setCount] = useState(0);
const { loading, error, data } = useFetch('/api');
```

---

20. Performance pitfalls of destructuring large objects?

Answer:

Each destructure expression:

- Creates a new binding environment
- Triggers hidden class transitions if overused in loops

Tip: Avoid destructuring large objects in tight loops or recursion.

💡 Real-World React / Dev Tips

Use Case	Pattern
Merge props	`{ ...defaultProps, ...userProps }`
Extract props	`const { id, onClick } = props;`
Combine states	`setState(prev => ({ ...prev, [key]: val }))`
Clone immutable state	`const newState = { ...state }`

⚠️ Common Pitfalls

1. Forgetting destructure defaults → `undefined` surprises.
2. Spreading nested objects — shallow copies break immutability.
3. Using rest at wrong place (must be last param/property).
4. Shadowing variables with destructure aliases.
5. Destructuring `null` / `undefined` without guards.

✅ Summary

Concept	Description
Spread	Expands iterable/object
Rest	Collects remaining args
Destructuring	Extracts properties/elements
Immutability	Spread avoids direct mutation
React Relevance	Used in props, hooks, immutability patterns

⚙️ Section 5.4 — Array Performance & Optimization (20 FAANG-Level Q&A)

---

1. What is the time complexity of common array operations?

Operation	Avg Time	Notes
`push`, `pop`	O(1)	Amortized constant
`shift`, `unshift`	O(n)	Re-indexes all elements
`splice`	O(n)	Depends on delete/insert count
`slice`, `concat`, `map`, `filter`, `reduce`	O(n)	Iterate entire array

✅ Use `push` / `pop` for stacks; avoid `shift` / `unshift` for large queues.

---

2. Why are `shift()` and `unshift()` slower than `push()` and `pop()`?

Because shifting modifies **index of every element**; `push` / `pop` operate only at the tail.

```js
const arr=[1,2,3]; arr.shift(); // moves 2→index0, 3→index1
```

---

3. How to optimize `map()` / `filter()` for large data?

- **Combine passes**: merge `filter` + `map` into one `reduce`.
- **Preallocate** if you know final length.
- For heavy compute, **use Web Workers** or batch process chunks.

---

4. Why prefer `for` or `for...of` over `forEach()` in perf-critical code?

`forEach` creates callback closures per iteration; `for` loops are monomorphic, easier for V8 to optimize.

```js
```

```
for(let i=0;i<items.length;i++) doWork(items[i]);
```

---

5. How does memory allocation affect array resizing in V8?

V8 uses **dynamic backing stores**: arrays grow exponentially. Repeated small pushes → occasional reallocations.
Pre-size with `new Array(n)` if length known.

---

6. What's the difference between sparse and dense arrays?

- **Dense** = contiguous numeric indexes.
- **Sparse** = missing indices or non-numeric keys.
  V8 optimizes dense arrays using packed elements; sparse arrays fall back to hash tables.

```js
const dense=[1,2,3];
const sparse=[]; sparse[1000]=1; // huge holes → slow
```

---

7. How do sparse arrays affect performance?

Sparse arrays disable inline element access optimizations → every lookup is slower (`O(1)` → hash lookup).

---

8. How can you preallocate arrays for better perf?

```js
const arr = new Array(100000).fill(0);
```

Pre-allocation minimizes reallocations, keeping array **packed**.

---

9. Why avoid `delete arr[i]`?

`delete` leaves holes (turns dense → sparse). Use `splice(i,1)` to remove and shift instead.

---

10. How to benchmark array methods properly?

Use:

```js
console.time('map');
arr.map(fn);
console.timeEnd('map');
```

or high-res timers: `performance.now()`.

Run multiple iterations to neutralize JIT warm-up.

---

11. How does V8 optimize monomorphic vs polymorphic arrays?

- **Monomorphic**: all elements same type → fast inline access.
- **Polymorphic**: mixed types (numbers + objects) → deoptimized.

```js
const arr=[1,2,3]; arr.push("x"); // changes hidden class
```

---

12. Convert array-like structures efficiently

Use modern `Array.from()` or spread:

```js
Array.from(nodeList);
[...arguments];
```

Both use internal optimized paths.

---

13. What is array de-optimization? When does it occur?

When JS engine invalidates compiled optimizations (e.g., switching element types, making sparse).
→ Performance drops to generic object lookups.

---

14. Why use Typed Arrays (`Int32Array` , `Float64Array`)?

They store values in contiguous binary memory → predictable access speed; ideal for numeric/ML workloads.

```js
const buf=new Float64Array(1e6);
```

---

15. How to minimize GC impact in heavy array ops?

- Reuse pre-allocated buffers.
- Avoid creating temporary arrays inside loops.
- Use `push` on one array rather than concatenating new ones.

---

16. Why are array indexes stored as strings internally?

In JS, objects are maps with string keys; array indexes are stringified numbers (`"0"` , `"1"`).
Engines add special “elements” optimization for numeric keys, but conceptually still string keys.

---

17. What is a packed array?

A **packed array** has no holes and uniform types.
Engines optimize access using **C-style contiguous memory**.
Creating a hole or type-mix unpacks it.

---

18. Flatten deep arrays without recursion overflow

Use `stack` or `reduce` iterative flatten:

```js
function flatten(arr){
  const res=[], stack=[...arr];
  while(stack.length){
    const next=stack.pop();
    Array.isArray(next)? stack.push(...next): res.push(next);
  }
  return res.reverse();
}
```

---

19. How to parallelize array transformations?

- Split work into chunks → send to **Web Workers**.
- Combine results when workers finish.

  Used for analytics, image filters, etc.

---

20. Best practices for large dataset filtering/pagination (frontend)

- Paginate on backend.
- Cache results client-side.
- Lazy-load or virtual-scroll large lists (React Window, React Virtualized).
- Debounce expensive operations.

⚡ Quick Performance Summary

Topic	Optimal Practice
Add/remove	Use `push` / `pop`, avoid `shift` / `unshift`
Copy	Use `slice` or spread
Transform	Combine passes (`reduce`)
Memory	Preallocate if size known
Data type	Keep arrays monomorphic

Topic	Optimal Practice
Large data	Stream/process in chunks

💡 Real-World / React View

Use Case	Perf Concern	Fix
`setState(prev => [...prev, newItem])`	New array per render	Fine for small sets; for large data, batch updates
`list.map()` in render	Creates new refs	Memoize (`useMemo`) or use keys efficiently
Virtual lists	Rendering large arrays	Use `react-window` , `react-virtualized`
Data analytics	Heavy reduce/filter	Move compute to Web Worker

---

🚫 Common Pitfalls

1. Mixing data types → hidden-class churn.
2. Deleting array items → creates sparse arrays.
3. Re-creating arrays in loops instead of mutating safely.
4. Using `JSON.parse/stringify` for deep copy (slow, loses types).
5. Relying on `forEach` in performance-critical paths.

✅ In Short

- Keep arrays **dense + monomorphic**.
- Combine transforms to reduce passes.
- Preallocate and reuse buffers.
- Use typed arrays for numeric tasks.
- Offload CPU-heavy ops to workers.

🧩 Section 5.5 — Custom Iterators (20 FAANG-Level Q&A + Answers)

---

1. How would you define a custom iterator for a plain object?

Answer:

You implement the **iterator protocol** by defining a `Symbol.iterator` method that returns an object with a `.next()` function.

```js
const obj = { a: 1, b: 2, c: 3 };

obj[Symbol.iterator] = function () {
  const keys = Object.keys(this);
  let i = 0;
  return {
    next: () => ({
      value: this[keys[i++]],
      done: i > keys.length,
    }),
  };
};

for (const val of obj) console.log(val); // 1, 2, 3
```

---

2. What is the purpose of `Symbol.iterator`?

It defines *how an object should be iterated* by constructs like:

- `for...of`
- Spread syntax `...`
- `Array.from()`

If an object has `[Symbol.iterator]`, it's **iterable**.

---

3. How can you make a custom data structure iterable with `for...of`?

By attaching `[Symbol.iterator]` that returns an iterator object.

Example: a custom **Range** class.

```js
class Range {
  constructor(start, end) {
    this.start = start; this.end = end;
  }
  [Symbol.iterator]() {
    let current = this.start;
    return {
      next: () => ({
        value: current,
        done: current++ > this.end,
      }),
    };
  }
}
for (const n of new Range(3, 5)) console.log(n); // 3,4,5
```

---

4. Implement a custom range iterator (like Python's `range()`)

```js
function range(start, end, step = 1) {
  return {
    [Symbol.iterator]() {
      let i = start;
      return {
        next() {
          if (i <= end) return { value: i += step - step, done: false };
          return { done: true };
        },
      };
    },
  };
```

```
}
for (const n of range(1, 3)) console.log(n); // 1,2,3
```

---

5. What's the difference between the iterator and iterable protocol?

Concept	Description
Iterable	Has `[Symbol.iterator]()` returning an iterator
Iterator	Has `.next()` returning `{ value, done }`

Some objects are both (like arrays, maps, sets).

---

6. How can you create an iterator that supports early termination with `return()`?

By adding a `return()` method to clean up.

```js
function makeIter() {
  let i = 0;
  return {
    next() { return { value: i++, done: i > 5 }; },
    return() { console.log("Stopped early!"); return { done: true }; }
  };
}
for (const n of makeIter()) {
  if (n === 3) break; // triggers return()
}
```

---

7. How does the `for...of` loop interact with manually defined `.next()`?

Each iteration:

1. Calls `.next()`
2. Checks `done`
3. Extracts `value`

4. Stops when `done:true`

---

8. How can you build an iterator that supports `break` / `continue` logic safely?

Ensure `.next()` always returns a valid `{ done, value }` pair — even if values are skipped.

```js
function evenIter(limit) {
  let n = 0;
  return {
    next() {
      while (n <= limit) {
        if (n++ % 2 === 0) return { value: n - 1, done: false };
      }
      return { done: true };
    },
  };
}
```

---

9. What happens if `.next()` doesn't return `{ done: true }` properly?

The iterator becomes **infinite** — `for...of` never ends, potentially freezing your app.

---

10. How can you implement bidirectional iterators (forward/backward)?

Return an object with both `.next()` and `.previous()`:

```js
function bidirectional(arr) {
  let i = 0;
  return {
    next() { return { value: arr[i++], done: i > arr.length }; },
    previous() { return { value: arr[--i], done: i < 0 }; },
  };
}
```

11. How can you combine generators and custom iterators?

Generators automatically implement `[Symbol.iterator]`, simplifying iteration.

```js
function* customRange(start, end) {
  for (let i = start; i <= end; i++) yield i;
}
for (const n of customRange(1, 3)) console.log(n);
```

12. Can you build an iterator that yields asynchronously (async iterators)?

Yes — by implementing `Symbol.asyncIterator`.

```js
const asyncRange = {
  from: 1, to: 3,
  [Symbol.asyncIterator]() {
    let i = this.from;
    return {
      next: () =>
        i <= this.to
          ? Promise.resolve({ value: i++, done: false })
          : Promise.resolve({ done: true }),
    };
  },
};

for await (const n of asyncRange) console.log(n);
```

13. What's the purpose of `Symbol.asyncIterator`?

Defines how asynchronous data (like streams or APIs) can be iterated with `for await...of`.

It enables **pull-based async iteration** over Promises.

14. How can you consume async iterators with `for await...of`?

```js
async function read() {
  for await (const chunk of asyncRange) console.log(chunk);
}
read();
```

---

15. How can iterators process streaming data efficiently?

Iterators allow **lazy data processing**:

```js
function* readLines(lines) {
  for (const l of lines.split('\n')) yield l.trim();
}
```

You only hold one line at a time → huge files handled safely.

---

16. How can you create a tree iterator (DFS traversal)?

```js
function* traverse(node) {
  if (!node) return;
  yield node.value;
  for (const child of node.children || []) yield* traverse(child);
}
```

---

17. How can you compose multiple iterators into one unified iterator?

```js
function* combine(...iters) {
  for (const it of iters) yield* it;
```

```
}
[...combine([1,2],[3,4])]; // [1,2,3,4]
```

---

18. What's the difference between iterators of `Map` / `Set` vs arrays?

- Arrays yield **values**
- Sets yield **values**
- Maps yield **[key, value] pairs**

```js
for (const [k,v] of new Map([['x',1]])) console.log(k,v);
```

---

19. How can you implement an infinite sequence generator with iterator pattern?

```js
function infiniteCounter(){
  let n = 0;
  return {
    next(){ return { value: n++, done: false }; }
  };
}
```

💡 Always limit consumption manually!

---

20. What are real-world use cases for custom iterators?

✅ Building **streaming parsers**
✅ Lazy-loading data structures
✅ Complex traversals (trees, graphs)
✅ Implementing custom pipelines (React Fiber)
✅ Database cursors / pagination wrappers

💡 Real-World React & Async Examples

Scenario	Iterator Role
Virtual scrolling	Lazy load items on demand
Pagination APIs	Async iterator over pages
React Fiber	Uses custom iterator-like traversal for rendering tree
Streaming JSON	Consume parts of API as they arrive
Data transforms	Generators as pipeline steps

🚫 Common Pitfalls

1. Returning invalid `{ value, done }` → crashes iteration
2. Forgetting to handle early termination (`return()`)
3. Not distinguishing between async & sync iterators
4. Consuming infinite iterators accidentally
5. Mixing iterables without proper `[Symbol.iterator]`

✅ Summary

Concept	Description
Iterable	Implements `[Symbol.iterator]()`
Iterator	Implements `.next()`
Async Iterable	Implements `[Symbol.asyncIterator]()`
Generator	Auto-creates iterator
Use Case	Lazy evaluation, streams, composable pipelines